

VUR 5

Combined Vulnerability & Security Findings Report

Total findings compiled from all sources: **36** (Critical: 2, High: 10, Medium: 18, Low: 6)

#	Severity	Finding	Location
1	Critical	Incomplete object-level authorization	src/server/routes/grievances.ts; src/server/routes/attachments.ts
2	Critical	Unsafe attachment filename handling	src/server/storage/attachments.ts
3	High	Plaintext session tokens stored in the database	session.ts
4	High	API traffic is served over HTTP without TLS enforcement	index.ts
5	High	Insecure password hashing (unsalted SHA-256)	src/server/auth/passwords.ts
6	High	Missing security flags on cookie (HttpOnly, Secure, SameSite)	src/server/auth/session.ts
7	High	Lack of rate limiting / brute-force protection on login API	src/server/routes/auth.ts
8	High	Session expiry is stored but not enforced	src/server/auth/session.ts
9	High	Logout does not delete the database session	src/server/auth/session.ts; routes/auth.ts
10	High	Overly permissive CORS configuration	src/server/app.ts
11	High	Stored XSS in comments	src/lib/components/app/comment-timeline.svelte:49
12	High	Comments are stored without sanitization	grievances.ts, comment creation endpoint
13	Medium	Unbounded text fields allow storage exhaustion	grievances.ts
14	Medium	Oversized uploads are fully loaded into memory	attachments.ts
15	Medium	Missing browser security headers	app.ts
16	Medium	File and database operations are not atomic	grievances.ts
17	Medium	Grievance listing endpoints have no pagination	grievances.ts and queries.ts
18	Medium	No attachment quota or count limit per grievance	grievances.ts
19	Medium	Database queries can be amplified through nested object assembly	queries.ts
20	Medium	Broken authorization on comment endpoints (GET & POST)	src/server/routes/grievances.ts
21	Medium	Unauthorized grievance status mutation by students	src/server/routes/grievances.ts
22	Medium	Race conditions & ID collisions in ID generator functions	src/server/db/queries.ts
23	Medium	Header injection & inline content-disposition XSS risks	src/server/routes/attachments.ts
24	Medium	Missing anti-CSRF token protections	src/server/app.ts & src/lib/services/api.ts
25	Medium	Weak upload validation	src/server/routes/grievances.ts
26	Medium	Detailed error messages returned to clients	src/server/http/errors.ts
27	Medium	Client-side localStorage state is trusted for navigation	src/lib/stores/auth.svelte.ts
28	Medium	Unlimited active sessions per account	auth.ts
29	Medium	Attachment downloads load the entire file into memory	attachments.ts

#	Severity	Finding	Location
30	Medium	No dependency lockfile or reproducible dependency versions	package.json
31	Low	No audit trail for grievance changes	grievances.ts
32	Low	No protection against database file exposure or weak file permissions	connection.ts and config.ts
33	Low	Hardcoded plaintext default credentials in seeder	src/server/db/seed.ts
34	Low	Development mock service bypasses authorization rules	mock.ts
35	Low	Destructive database reset has no environment safeguard	reset.ts and reset-db.ts
36	Low	Health endpoint reveals service availability without authentication	app.ts

Critical Severity Findings

1. Incomplete object-level authorization

Severity: Critical

Location: src/server/routes/grievances.ts; src/server/routes/attachments.ts

Issue

Students may access or modify other students' grievance data if IDs are known.

Fix

Verify resource ownership server-side on every read/write, not just authentication.

2. Unsafe attachment filename handling

Severity: Critical

Location: src/server/storage/attachments.ts

Issue

Crafted filenames may potentially write files outside the upload directory (path traversal).

Fix

Sanitize and normalize filenames, generate server-side names, and confine writes to the upload root.

High Severity Findings

3. Plaintext session tokens stored in the database

Severity: High

Location: session.ts

Issue

Session bearer tokens are stored directly in the sessions table. If the database or a backup is accessed, attackers can reuse active tokens.

Fix

Store only hashed session tokens and compare hashes when authenticating.

4. API traffic is served over HTTP without TLS enforcement

Severity: High

Location: index.ts

Issue

The server starts on http://127.0.0.1 with no HTTPS redirect or secure deployment requirement. On a networked deployment, login credentials and session cookies could be intercepted.

Fix

Serve behind HTTPS, redirect HTTP to HTTPS, enable HSTS, and ensure secure proxy configuration.

5. Insecure password hashing (unsalted SHA-256)

Severity: High

Location: src/server/auth/passwords.ts

Issue

Passwords are hashed with unsalted SHA-256, which is fast to brute-force and vulnerable to rainbow-table attacks.

Fix

Use a slow, salted password hashing algorithm such as bcrypt, scrypt, or Argon2.

6. Missing security flags on cookie (HttpOnly, Secure, SameSite)

Severity: High

Location: src/server/auth/session.ts

Issue

Session cookies are set without HttpOnly, Secure, or SameSite flags, exposing them to theft via XSS or man-in-the-middle attacks.

Fix

Set HttpOnly, Secure, and SameSite=Strict/Lax on all session cookies.

7. Lack of rate limiting / brute-force protection on login API

Severity: High

Location: src/server/routes/auth.ts

Issue

The login endpoint has no rate limiting, allowing unlimited password-guessing attempts.

Fix

Add rate limiting, exponential backoff, and account lockout or CAPTCHA after repeated failures.

8. Session expiry is stored but not enforced

Severity: High

Location: src/server/auth/session.ts

Issue

Sessions may remain valid beyond their intended seven-day expiry because the stored expiry value is not checked on each request.

Fix

Validate session expiry on every authenticated request and reject expired sessions.

9. Logout does not delete the database session

Severity: High

Location: src/server/auth/session.ts; routes/auth.ts

Issue

Copied session tokens may remain usable after logout because the session record is not removed from the database.

Fix

Delete the session record server-side on logout, not just the client-side cookie.

10. Overly permissive CORS configuration

Severity: High

Location: src/server/app.ts

Issue

Credentialed cross-origin requests may expose the API to unintended origins.

Fix

Restrict CORS to an explicit allow-list of trusted origins and avoid wildcard origins with credentials.

11. Stored XSS in comments

Severity: High

Location: src/lib/components/app/comment-timeline.svelte:49

Issue

Comment bodies are rendered raw via an unescaped HTML directive, allowing injected HTML/JavaScript to execute in other users' browsers.

Fix

Render comment bodies as plain text (remove the raw-HTML directive) or sanitize HTML server-side before storage/render.

12. Comments are stored without sanitization

Severity: High

Location: grievances.ts, comment creation endpoint

Issue

Comment content is saved as-is with no sanitization, compounding the stored-XSS risk above.

Fix

Sanitize comment content on the server before saving, or reject HTML characters and tags.

Medium Severity Findings

13. Unbounded text fields allow storage exhaustion

Severity: Medium

Location: grievances.ts

Issue

Titles, descriptions, and comments have minimum-length checks but no maximum lengths. Attackers can submit extremely large values, increasing memory usage and database size.

Fix

Enforce maximum lengths at the API and database boundaries.

14. Oversized uploads are fully loaded into memory

Severity: Medium

Location: attachments.ts

Issue

The complete upload is read into memory before the 2 MB size check. Repeated oversized uploads could cause memory exhaustion.

Fix

Enforce request limits and stream uploads with a strict byte cap.

15. Missing browser security headers

Severity: Medium

Location: app.ts

Issue

Security headers such as CSP, X-Content-Type-Options, Referrer-Policy, and frame protections are not configured.

Fix

Add security-header middleware and a restrictive Content Security Policy.

16. File and database operations are not atomic

Severity: Medium

Location: grievances.ts

Issue

Files are written separately from database records. A failure between operations can create orphaned files or database records pointing to missing files, potentially exhausting disk space.

Fix

Use database transactions, temporary files, cleanup on failure, and startup reconciliation.

17. Grievance listing endpoints have no pagination

Severity: Medium

Location: grievances.ts and queries.ts

Issue

The API returns every grievance, including all comments and attachments, in one response. As records grow, an authenticated user can trigger large memory usage, slow database queries, and oversized responses.

Fix

Add pagination, maximum page sizes, filtering, and database-level limits.

18. No attachment quota or count limit per grievance

Severity: Medium

Location: grievances.ts

Issue

Students can repeatedly upload attachments to the same unresolved grievance. Each file can be up to 2 MB, allowing unlimited disk consumption over time.

Fix

Enforce per-user, per-grievance, and global storage quotas, and limit the number of attachments.

19. Database queries can be amplified through nested object assembly

Severity: Medium

Location: queries.ts

Issue

Listing many grievances performs additional queries for each grievance's student, comments, comment authors, and attachments (an N+1 query pattern) that can become a database denial-of-service problem as data grows.

Fix

Use joined or batched queries, pagination, and query-count performance tests.

20. Broken authorization on comment endpoints (GET & POST)

Severity: Medium

Location: src/server/routes/grievances.ts

Issue

Comment read/write endpoints do not properly verify that the requesting user is entitled to view or post on the target grievance.

Fix

Add ownership/role checks to both the GET and POST comment handlers before processing.

21. Unauthorized grievance status mutation by students

Severity: Medium

Location: src/server/routes/grievances.ts

Issue

Students can change a grievance's status directly, a privilege that should be restricted to staff/admin roles.

Fix

Enforce role-based authorization on the status-update endpoint, restricting it to authorized staff.

22. Race conditions & ID collisions in ID generator functions

Severity: Medium

Location: src/server/db/queries.ts



Issue

ID generation is not safe under concurrent requests, risking duplicate or colliding identifiers.

Fix

Use database-generated IDs (auto-increment/UUID) or a properly locked/atomic ID generation strategy.

23. Header injection & inline content-disposition XSS risks

Severity: Medium

Location: src/server/routes/attachments.ts

Issue

Attachment filenames/metadata are placed into response headers (e.g. Content-Disposition) without sanitization, risking header injection or reflected XSS on download.

Fix

Sanitize/encode filenames before placing them in headers and enforce a strict header-value allow-list.

24. Missing anti-CSRF token protections

Severity: Medium

Location: src/server/app.ts & src/lib/services/api.ts

Issue

State-changing requests do not require a CSRF token, allowing forged cross-site requests from an authenticated user's browser.

Fix

Implement anti-CSRF tokens (or SameSite cookies plus origin checks) on all state-changing endpoints.

25. Weak upload validation

Severity: Medium

Location: src/server/routes/grievances.ts

Issue

Client-controlled MIME types can be spoofed and over-sized bodies may consume resources.

Fix

Validate file content/type server-side (magic-byte checks) and enforce strict size limits before processing.

26. Detailed error messages returned to clients

Severity: Medium

Location: src/server/http/errors.ts

Issue

Database or filesystem details may be exposed to clients through verbose error responses.

Fix

Return generic error messages to clients and log detailed errors server-side only.

27. Client-side localStorage state is trusted for navigation

Severity: Medium

Location: src/lib/stores/auth.svelte.ts

Issue

Manipulated or stale UI identity information may persist and influence navigation/authorization decisions client-side.

Fix

Treat client-stored identity as UI-only; always re-verify authorization server-side for protected actions.

28. Unlimited active sessions per account

Severity: Medium

Location: auth.ts

Issue

Every successful login creates a new session, but there is no per-user session limit or session management. An attacker with valid credentials can create thousands of sessions and grow the sessions table.

Fix

Limit active sessions per user, revoke older sessions, and periodically remove unused session records.

29. Attachment downloads load the entire file into memory

Severity: Medium

Location: attachments.ts

Issue

Downloading an attachment uses `readFileSync`, which loads the entire file into memory before sending it. Many concurrent downloads can exhaust server memory even when each file is within the upload limit.

Fix

Stream files using a read stream and apply download concurrency and bandwidth controls.

30. No dependency lockfile or reproducible dependency versions

Severity: Medium

Location: package.json

Issue

Dependencies use version ranges such as `^4.13.5`, and no lockfile is present in the workspace. A fresh installation may silently receive a compromised or vulnerable dependency update.

Fix

Commit `package-lock.json`, use automated dependency scanning, and review dependency updates before deployment.

Low Severity Findings

31. No audit trail for grievance changes

Severity: Low

Location: grievances.ts

Issue

Status changes and grievance edits do not record the acting user, previous value, new value, or action time in an audit table.

Fix

Add an append-only audit log for all privileged and user-generated changes.

32. No protection against database file exposure or weak file permissions

Severity: Low

Location: connection.ts and config.ts

Issue

The application creates the SQLite database and upload directory using default operating-system permissions. On a shared host, other local users or processes may be able to read sensitive grievance data and session records.

Fix

Apply restrictive file and directory permissions, isolate the service account, and protect database backups and WAL files.

33. Hardcoded plaintext default credentials in seeder

Severity: Low

Location: src/server/db/seed.ts

Issue

The database seed script contains hardcoded plaintext default credentials, which may be left active in a deployed environment.

Fix

Remove hardcoded credentials from seed scripts, or force a mandatory password reset on first login for seeded accounts.

34. Development mock service bypasses authorization rules

Severity: Low

Location: mock.ts

Issue

The mock `updateStatus` method changes any grievance without checking the current user's role or ownership. The mock `comment` method also trusts the caller-provided `authorId`. If the mock service is accidentally used in a deployed build, users could modify records or impersonate comment authors.

Fix

Enforce authorization inside the mock service too, and use an explicit production build configuration that cannot select mock services.

35. Destructive database reset has no environment safeguard

Severity: Low

Location: reset.ts and reset-db.ts

Issue

Running the reset command deletes the database, WAL files, and uploads directory without checking whether the environment is development or production. An accidental production execution would permanently destroy application data.

Fix

Require an explicit development environment flag and confirmation token before allowing a reset.

36. Health endpoint reveals service availability without authentication

Severity: Low

Location: app.ts

Issue

`/api/health` is publicly accessible and allows anyone who can reach the server to probe whether the service is running. This assists automated service discovery and availability monitoring by attackers.

Fix

Restrict health checks to an internal network or expose only a minimal liveness endpoint through the reverse proxy.
